

Mise à niveau Linux

Les bases pour l'apprenti administrateur système

Dr. Maïssa MBAYE

maïssa.mbaye@ugb.edu.sn

**Master Ingénierie Informatique et Technologies de l'Information et de la
Communication (2ITIC)**



Université Gaston Berger de Saint-Louis (UGB)
Section informatique, UFR SAT

Année académique 2016-2017

Première partie I

Linux de tous les jours

- 1 Shell, commandes et aide en ligne
 - Le Shell
 - Commandes et Aide en ligne
 - Variables d'environnement
 - Caractères spéciaux du Shell
 - Entrées/Sorties et redirection

- 1 Shell, commandes et aide en ligne
 - Le Shell
 - Commandes et Aide en ligne
 - Variables d'environnement
 - Caractères spéciaux du Shell
 - Entrées/Sorties et redirection

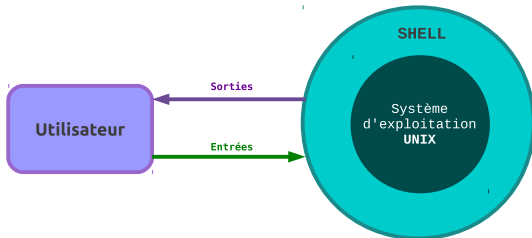
Shell Unix

Idée de Shell

Un Shell ^a est :

- Un programme qui permet aux utilisateurs de lancer d'autres programmes
- Une couche logicielle servant d'interface entre les utilisateurs et le système d'exploitation Unix
 - Une des innovations majeures d'Unix a été la séparation du système d'exploitation et du Shell

a. cocquille ou cocque en français



Shell Unix

Il existent deux types de Shells :

- **Shells en mode texte :**

- Entrées sous forme de textes (les commandes)
- Sorties textuelles formatées ou non
- Utilisation orientée administration
- Autres appellations : Terminal, Console, ...

- **Shells graphiques :**

- Entrées sous plusieurs formes : texte, souris, touch, ...
- Sorties aux formats quelconque : fenêtres, ...
- Utilisations comme ordinateur personnel (bureautique, ...)
- Autres appellations : GUI, Gestionnaire de Bureau

L'emploi du mot **Shell** sous Unix, sans autre précision, fait généralement référence aux **Shells** orientés texte (pour des raisons historiques...)



Les Shells Linux

- **Bourne Shell** ou `sh`
 - Porte le nom de son créateur Steven Bourne
 - correspond au programme `sh`
- **C shell** ou `csh`
 - Shell dont la syntaxe du langage de script est proche du langage C
- **GNU Bourne-Again SHell** ou `bash`
 - Une version GNU du Bourne Shell
 - Le Shell le plus répandu dans les distributions Linux
- ...

la liste des Shells installés sur station Unix est disponible avec la commande :

```
> cat /etc/shells
```



Emulateurs de Shell ou terminal

Les environnements graphiques de Linux proposent des émulateurs de Shell : Xterm, Konsole, Gnome-terminal, ...

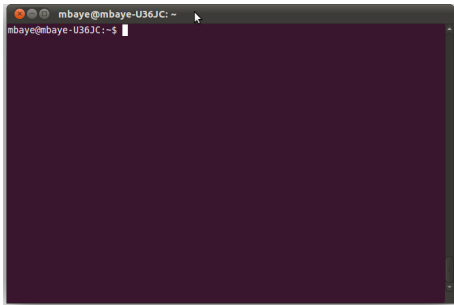


Figure: Gnome Terminal



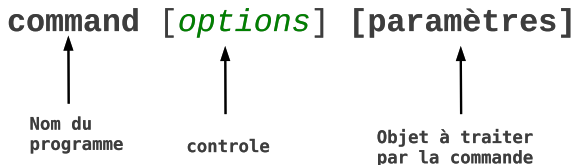
Figure: Xterm



- 1 Shell, commandes et aide en ligne
 - Le Shell
 - Commandes et Aide en ligne**
 - Variables d'environnement
 - Caractères spéciaux du Shell
 - Entrées/Sorties et redirection

Qu'est ce qu'une commande Unix ?

une commande est un programme réalisant une tâche spécifique et pouvant être exécutée par un utilisateur (au sens large)



Exemple : `ls -l /home/sroot/`

- `ls` est la commande (que faire ?)
- `-l` est une option (comment faire ?)
- `/home/sroot/` est le paramètre transmis à la commande (sur quoi faire ?)



Qu'est ce qu'une commande Unix ?

Le resultat de l'exécution d'une commande est :

- soit un affichage de texte
- soit une modification du système
- soit les deux

Exemple de sortie

```
~:> ls -l /home/sroot/
total 44
drwxr-xr-x 2 sroot sroot 4096 janv.  7 18:44 Bureau
drwxr-xr-x 2 sroot sroot 4096 janv.  7 18:44 Documents
-rw-r-r-  1 sroot sroot 8445 janv.  7 14:22 examples.desktop
drwxr-xr-x 2 sroot sroot 4096 janv.  7 18:44 Images
drwxr-xr-x 2 sroot sroot 4096 janv.  7 18:44 Modèles
...
```



Types de commandes et la commande <type>

Il existe trois types de commandes

- Les commandes internes au Shell (elles sont installées en même temps que le Shell)
- Les commandes externes au Shell : installées par l'utilisateur ou faisant partie de la distribution
- Les alias qui sont des pseudos commandes basées sur les autres commandes existantes

↔ La commande **type** permet de connaître le type d'une commande

```
user@machine:~> type ls
ls est un alias vers « ls --color=auto »
```

```
user@machine:~> type bg
bg est une primitive du shell
```

```
user@machine:~> type passwd
passwd est /usr/bin/passwd
```



Options des commandes

- Les options des commandes permettent à l'utilisateur de contrôler leurs comportements
- L'ordre des options n'est normalement pas important, doivent être avant le paramètre

Exemple :

```
user@machine:~> ncal
```

Janvier 2013

lu		7	14	21	28	
ma	1	8	15	22	29	
me	2	9	16	23	30	
je	3	10	17	24	31	
ve	4	11	18	25		
sa	5	12	19	26		
di	6	13	20	27		

```
user@machine:~> ncal -b
```

Janvier 2013

lu	ma	me	je	ve	sa	di
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			



Options des commandes

Il existe quatre types d'options suivant la forme :

- **Type Unix** : introduit par Unix System V
- **Type BSD** : introduit par Unix BSD
- **Type Long GNU** : introduit par le mouvement GNU
- **Type ambigu** : nom donné par Richard Stallman aux options qui ne collent à aucun des types précédents



Options des commandes

- Option type Unix : Caractère(s) alphanumérique(s) précédé(s) d'un tiret (-)



Options des commandes

- Option type Unix : Caractère(s) alphanumérique(s) précédé(s) d'un tiret (-)

Exemple

```
> ls -l -h repertoire  
> ls -la repertoire    #combinaison de deux options -l et -a
```



Options des commandes

- Option type Unix : Caractère(s) alphanumérique(s) **précédé(s) d'un tiret (-)**

Exemple

```
> ls -l -h repertoire  
> ls -la repertoire    #combinaison de deux options -l et -a
```

- Option type BSD : Caractère(s) alphanumérique(s) **sans tiret (-)**



Options des commandes

- Option type Unix : Caractère(s) alphanumérique(s) **précédé(s) d'un tiret (-)**

Exemple

```
> ls -l -h repertoire  
> ls -la repertoire #combinaison de deux options -l et -a
```

- Option type BSD : Caractère(s) alphanumérique(s) **sans tiret (-)**

Exemple

```
> tar xvzf archive.tar.gz #combinaison des options x,v,z et -f  
> ps aux #combinaison des options a,u et x
```



Options des commandes

- Option type Unix : Caractère(s) alphanumérique(s) **précédé(s) d'un tiret (-)**

Exemple

```
> ls -l -h repertoire
> ls -la repertoire    #combinaison de deux options -l et -a
```

- Option type BSD : Caractère(s) alphanumérique(s) **sans tiret (-)**

Exemple

```
> tar xvzf archive.tar.gz #combinaison des options x,v,z et -f
> ps aux #combinaison des options a,u et x
```

- Option longue type GNU : suite de mots (souvent en anglais) dont **le premier est précédé d'un double tiret (--)** et **les autres sont séparés par des tirets (-)**



Options des commandes

- Option type Unix : Caractère(s) alphanumérique(s) **précédé(s) d'un tiret (-)**

Exemple

```
> ls -l -h repertoire  
> ls -la repertoire #combinaison de deux options -l et -a
```

- Option type BSD : Caractère(s) alphanumérique(s) **sans tiret (-)**

Exemple

```
> tar xvzf archive.tar.gz #combinaison des options x,v,z et -f  
> ps aux #combinaison des options a,u et x
```

- Option longue type GNU : suite de mots (souvent en anglais) dont **le premier est précédé d'un double tiret (--)** et **les autres sont séparés par des tirets (-)**

Exemple

```
> ls --all --human-readable --author un-repertoire # trois options
```



Options des commandes

- Option type Unix : Caractère(s) alphanumérique(s) **précédé(s) d'un tiret (-)**

Exemple

```
> ls -l -h repertoire
> ls -la repertoire #combinaison de deux options -l et -a
```

- Option type BSD : Caractère(s) alphanumérique(s) **sans tiret (-)**

Exemple

```
> tar xvzf archive.tar.gz #combinaison des options x,v,z et -f
> ps aux #combinaison des options a,u et x
```

- Option longue type GNU : suite de mots (souvent en anglais) dont **le premier est précédé d'un double tiret (--)** et **les autres sont séparés par des tirets (-)**

Exemple

```
> ls --all --human-readable --author un-repertoire # trois options
```

- Option longue type ambiguë : Options dont il est difficile de savoir si c'est une combinaison d'options ou une option longue



Options des commandes

- Option type Unix : Caractère(s) alphanumérique(s) **précédé(s) d'un tiret (-)**

Exemple

```
> ls -l -h repertoire
> ls -la repertoire #combinaison de deux options -l et -a
```

- Option type BSD : Caractère(s) alphanumérique(s) **sans tiret (-)**

Exemple

```
> tar xvzf archive.tar.gz #combinaison des options x,v,z et -f
> ps aux #combinaison des options a,u et x
```

- Option longue type GNU : suite de mots (souvent en anglais) dont **le premier est précédé d'un double tiret (--)** et **les autres sont séparés par des tirets (-)**

Exemple

```
> ls --all --human-readable --author un-repertoire # trois options
```

- Option longue type ambiguë : Options dont il est difficile de savoir si c'est une combinaison d'options ou une option longue

Exemple

```
> find . -name "*.avi" #combinaison des options n,a,m et e ou bien une seule option --name?
```



Alias de commande : commande <alias>

Les alias sont des raccourcis d'autres commandes

- Permettent de créer simplement de nouvelles commandes en se basant sur les commandes existantes
- Les alias créées sur la ligne de commande ne sont valables que sur les terminaux sur lesquels ils sont définis.

↪ La commande **alias** permet de créer des alias

Exemple

```
> ls -l
total 16
drwxrwxr-x 2 sroot sroot 4096 janv. 21 10:37 bin
drwxrwxr-x 2 sroot sroot 4096 janv. 21 10:37 include
drwxrwxr-x 3 sroot sroot 4096 janv. 21 10:37 lib
drwxrwxr-x 4 sroot sroot 4096 janv. 21 10:37 src
> alias ll='ls -l'
> alias ll
alias ll='ls -l'
> ll
total 16
drwxrwxr-x 2 sroot sroot 4096 janv. 21 10:37 bin
drwxrwxr-x 2 sroot sroot 4096 janv. 21 10:37 include
drwxrwxr-x 3 sroot sroot 4096 janv. 21 10:37 lib
drwxrwxr-x 4 sroot sroot 4096 janv. 21 10:37 src
...
```



Documentation en ligne

- ❶ La commande **man** permet de présenter la documentation qui :
 - est installée en même temps que la commande et qui fait office de mode d'emploi
- ❷ La commande **info** permet d'accéder à des tutoriels sur l'utilisation des commandes :
 - Contenu plus orienté pédagogie
- ❸ La commande **help** permet d'accéder à l'aide des commandes internes au Shell
- ❹ La commande **apropos** cherche le nom et la description des pages de manuel
- ❺ Dans les environnement graphiques (Gnome, KDE, Xfce, ...) la touche [F1] donne accès à de la documentation



Commande man

Syntaxe de la commande

```
man [section] [nom_commande]
```

Extrait de la page manuelle de la commande man

```
...
Le tableau ci-dessous indique le numéro des sections de manuel ainsi
que le type de pages qu'elles contiennent.
1 Programmes exécutable ou commandes de l'interpréteur de commandes (shell);
2 Appels système (fonctions fournies par le noyau);
3 Appels de bibliothèque (fonctions fournies par les bibliothèques des programmes);
4 Fichiers spéciaux (situés généralement dans /dev);
5 Formats des fichiers et conventions. Par exemple /etc/passwd;
6 Jeux;
7 Divers (y compris les macropaquets et les conventions). Par exemple, man(7), groff(7);
8 Commandes de gestion du système (généralement réservées au superutilisateur);
9 Sous-programmes du noyau [hors standard].
...
```



Commande man

Utilisation de base :

```
> man printf
> man 3 printf
> man -f printf
```

Recherche de page manuelle :

```
> man -k "list directory"
dir (1)          - list directory contents
ls (1)           - list directory contents
vdir (1)         - list directory contents
> apropos "list directory"
dir (1)          - list directory contents
ls (1)           - list directory contents
vdir (1)         - list directory contents
> man -f printf
printf (3)       - formatted output conversion
printf (1)       - format and print data
```

La commande **mandb** reconstruit la base de données des pages manuelles



Commande man

> man 3 printf

```

mbaye@mbaye-U36JC: ~/SupportDeCoursSurData/Introduction à Linux (commande et système)/beamer-presentation/img
PRINTF(3)                                Linux Programmer's Manual                                PRINTF(3)

NAME
    printf, fprintf, sprintf, snprintf, vprintf, vfprintf, vsprintf, vsnprintf - formatted output conversion

SYNOPSIS
    #include <stdio.h>

    int printf(const char *format, ...);
    int fprintf(FILE *stream, const char *format, ...);
    int sprintf(char *str, const char *format, ...);
    int snprintf(char *str, size_t size, const char *format, ...);

    #include <stdarg.h>

    int vprintf(const char *format, va_list ap);
    int vfprintf(FILE *stream, const char *format, va_list ap);
    int vsprintf(char *str, const char *format, va_list ap);
    int vsnprintf(char *str, size_t size, const char *format, va_list ap);

    Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

    snprintf(), vsnprintf():
        _BSD_SOURCE || _XOPEN_SOURCE >= 560 || _ISOC99_SOURCE || _POSIX_C_SOURCE >= 200112L;
        or cc -std=c99

DESCRIPTION
    The functions in the printf() family produce output according to a format as described below. The functions
    printf() and vprintf() write output to stdout, the standard output stream; fprintf() and vfprintf() write output to
    the given output stream; sprintf(), snprintf(), vsprintf() and vsnprintf() write to the character string str.

    The functions snprintf() and vsnprintf() write at most size bytes (including the trailing null byte ('\0')) to str.

    The functions vprintf(), vfprintf(), vsprintf(), and vsnprintf() are equivalent to the functions printf(), fprintf(),
    sprintf(), and snprintf(), respectively, except that they are called with a va_list instead of a variable number of
    arguments. These functions do not call the va_arg macro. Because they invoke the va_arg macro, the value of ap is
    undefined after the call. See stdarg(3).

    These eight functions write the output under the control of a format string that specifies how subsequent arguments
    (or arguments accessed via the variable-length argument facilities of stdarg(3)) are converted for output.

Manual page printf(3) line 1
    
```



La commande help

la commande `help` permet d’Afficher des informations sur les commandes intégrées dans `bash` (voir `man bash`)

Exemple

```
> help bg
```

```
bg: bg [job_spec ...]
```

Déplace des tâches vers l’arrière plan.

Place chaque `JOB_SPEC` en arrière plan comme s’il avait été démarré avec « `&` ».

Si `JOB_SPEC` n’est pas fourni, le shell utilise sa propre notion de tâche actuelle.

Code de sortie :

Renvoie le code de succès à moins que le contrôle de tâche ne soit pas activé ou qu’une erreur ne survienne.



Documentation **Linux** sur Internet

HOWTO (Comment faire)

Les HOWTOs sont des tutoriels sur des aspects spécialisés de Linux et bien plus ...

- Le site <http://www.tldp.org/> (The Linux Documentation Project) est une base de données de HOWTOs

Des milliers de sites sont dédiés au système d'exploitation **Linux**

- <http://www-106.ibm.com/developerworks/linux/> : IBM developerWorks
- <http://librenix.com/> : Librenix
- <http://www.tuxmobil.org/howtos.html> TuxMobile
- ...

Des magazines de références :

- Linux Journal
- Linux Magazine
- SysAdmin Magazine
- Unix Review
- ...



- 1 Shell, commandes et aide en ligne
 - Le Shell
 - Commandes et Aide en ligne
 - **Variables d'environnement**
 - Caractères spéciaux du Shell
 - Entrées/Sorties et redirection

Caractéristiques

Définition

- Les variables d'environnements sont des variables du Shell qui stockent des informations sur le système et de l'environnement de travail
- Elles sont utilisées pour avoir des informations partagées entre les différents programmes s'exécutant sur le Shell

Deux catégories de variables d'environnements selon la portée :

- Variables globales :
 - Visibles sur le Shell de session ainsi que tous les processus engendrés par ce dernier
 - la commande `printenv` permet d'afficher les variables d'environnement globales
- Variables locales :
 - locales au processus qui les a défini
 - Les variables locales sont la différence entre les sortie de `set` et `printenv`



Quelques variables d'environnement

\$HOME ou **~** : La variable qui contient le chemin du répertoire de l'utilisateur courant

```
> echo $HOME  
/home/toto
```

\$PATH : liste de repertoire où il faut aller chercher les commandes tapées

```
> echo $PATH  
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games
```

\$USER : login de l'utilisateur courant

\$PWD et **\$OLDPWD** : repertoire courant et repertoire precedent

```
> echo $PWD  
/home/toto/unrepertoire
```

\$PS1 et **\$PS2** : caractère de prompt du Shell ainsi ce que celui des commandes sur plusieurs lignes



Création de variable d'environnement **locale**

- **Création d'une variable d'environnement locale** $\Rightarrow$ simple affectation

```
nomvariable=[ Valeur | 'valeur avec espace' ]
```

- **Accès à la valeur de la variable par le caractère \$ devant**

```
echo $nomvariable
```

```
#création de variable
```

```
> nomvariable="Valeur de la variable"
```

```
> echo $nomvariable
```

```
Valeur de la variable
```

```
#Mauvaises définitions
```

```
#Espaces autour du signe =
```

```
> nomvariable = "hello world"
```

```
nomvariable : commande introuvable
```

```
> nomvariable= "hello world"
```

```
> echo $nomvariable
```

```
#Absence de quotes autour des valeurs avec espaces
```

```
> nomvariable= hello world
```

```
Commande «hello» non trouvée
```



Création de variable d'environnement **globale**

- **Création par la commande `export`**
`export [nom[=valeur] ...]`
- **Valable pour le Shell de création ainsi que les sous-Shells**

#création de variable

```
> export nomvariable="Valeur de la variable"  
> echo $nomvariable  
Valeur de la variable
```

```
> variable="Hello World"  
> printenv | grep vari  
> export variable="Hello World"  
> printenv | grep vari  
variable=Hello World  
> export -p | grep varia  
declare -x variable="Hello World"
```

#Affichage des variables définies par export

```
> export -p | head -n 5  
declare -x COLORTERM="gnome-terminal"  
declare -x COMPIZ_CONFIG_PROFILE="ubuntu"  
...  
declare -x DEFAULTS_PATH="/usr/share/gconf/gnome.default.path"  
declare -x DESKTOP_SESSION="gnome"
```



Suppression de variable d'environnement : unset

- La commande **unset** permet de supprimer une variable
`nom [-p|-v] [nom[=valeur] ...]`

```
> echo $variable  
Hello World  
> unset variable  
> echo $variable
```



Persistence d'une variable d'environnement

- Les variables (créées ou modifiées) ne survivent pas à la fin de la session Shell
- **Solution** : Mettre la définition de la variable dans l'un des fichiers
 - Consultés au démarrage du Shell :
 - `$HOME/.bash_profile`
 - `$HOME/.bash_login`
 - `$HOME/.profile`
 - Au moment de la session :
 - `$HOME/.bashrc`
- **Mise à jour** : la commande **source** et son alias **."** permettent de forcer le Shell à consulter un fichier de script

Ces commandes font la même chose

```
> source ~/.bashrc
> . ~/.bashrc
> source $HOME/.bashrc
> . $HOME/.bashrc
```



- 1 Shell, commandes et aide en ligne
 - Le Shell
 - Commandes et Aide en ligne
 - Variables d'environnement
 - **Caractères spéciaux du Shell**
 - Entrées/Sorties et redirection

Caractères spéciaux et wildcards

Les Wildcards sont des caractères spéciaux qui permettent de spécifier de manière rapide le nom d'un ensemble de fichiers.

Wildcard	Signification
*	Chaine de caractères quelconque
?	Un (unique) caractère quelconque
[caracteres]	N'importe quel caractère dans l'ensemble caracteres
[!caracteres]	N'importe quel caractère qui n'est pas dans l'ensemble caracteres
[:alnum:]	Un caractère alphanumérique quelconque
[:alpha:]	Un caractère alphabétique
[:digit:]	Un nombre
[:lower:]	Une lettre en minuscule
[:upper:]	Une lettre en majuscules



Caractère Astérisk(*)

- Le caractère Astérisk ou étoile (*) signifie une séquence quelconque de caractères
- Un nom qui contient le caractère * représente un pattern dont la partie (*) peut être remplacée par n'importe quelle chaîne de caractères

Exemples

- Affichage de tous les fichiers commençant par "syslog"

```
> cat syslog*
```
- Affichage de l'ensemble des mots contenant les caractères u, g et b dans l'ordre

```
> grep -e *u*g*b* dictionnaire.txt
```
- Affichage des fichiers musicaux

```
> ls *.mp3 *.wma
```
- Affichage des fichiers cachés du repertoire courant

```
> ls .*
```
- Affichage de quelques fichiers de configurations de Linux

```
> ls /etc/*.conf
```



Le wildcard point d'interrogation (?) :

- Le wildcard (?) signifie un caractère (seulement) quelconque
- Est utilisé souvent lorsque la différence entre les cibles est effectuée sur un caractère

Exemples

- Fichiers dont le nom commence par `article1` suivi d'un caractère
`> ls article1?`
- Fichiers PDF ayant deux caractères dans le nom avant l'extension
`> ls ?? .pdf`
- Fichiers ayant une extension de trois caractères
`> ls *.???`



Le wildcard crochets ([])

- Le wildcard [] signifie un caractère (seulement) dans l'ensemble entre les crochets
- Ensembles remarquables [0-9], [a-z], [A-Z], [a-c, u-z], ...

Exemples

- Entrées du dictionnaire entre A et C (casse ignorée)


```
> ls [a-c, A-C]*?
```
- Photos de l'UGB entre 2000 et 2009


```
> ls UGB200[0-9].jpg
```
- Fichiers avec seulement deux caractères alphabétiques


```
> ls [a-z, A-Z][a-z, A-Z]
```



Les listes : {}

- Les accolades {} permettent de créer une liste d'éléments séparés par des virgules ","
- Les accolades peuvent être imbriquées pour avoir une liste comme element dans une liste

Exemples

```
> ls -ld /etc/rc{0,1,2,3,4,5,6}.d
drwxr-xr-x 2 root root 4096 janv.  8 18:53 /etc/rc0.d
drwxr-xr-x 2 root root 4096 janv.  8 18:53 /etc/rc1.d
drwxr-xr-x 2 root root 4096 janv.  8 18:53 /etc/rc2.d
drwxr-xr-x 2 root root 4096 janv.  8 18:53 /etc/rc3.d
drwxr-xr-x 2 root root 4096 janv.  8 18:53 /etc/rc4.d
drwxr-xr-x 2 root root 4096 janv.  8 18:53 /etc/rc5.d
drwxr-xr-x 2 root root 4096 janv.  8 18:53 /etc/rc6.d
> mkdir -p /opt/weblogic/domains/domain{1,2,3}/bin
> mkdir -p {src/{iofunct,pluggins},bin,lib/mpi,include}
> tree .
.
-- bin
-- include
-- lib
  |-- mpi
-- src
  |-- iofunct
  |-- pluggins
7 directories, 0 files
```



Le tild (`~`) ou le repertoire `$HOME`

- Le caractère "`~`" est un l'équivalent du répertoire de l'utilisateur connecté
- Il peut être utilisé dans les chemins des fichiers

Exemples

```
> ls ~  
Bureau  Documents  examples.desktop  Images  
Modèles Musique  Public Téléchargements  
test  Vidéos  
> ls ~/test  
bin include lib src  
> dirname ~/Documents/  
/home/sroot
```



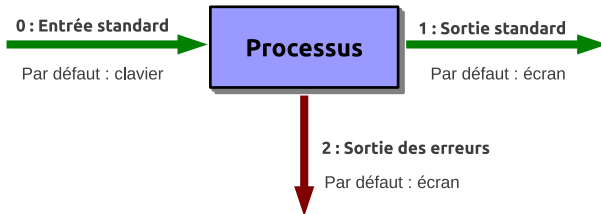
- 1 Shell, commandes et aide en ligne
 - Le Shell
 - Commandes et Aide en ligne
 - Variables d'environnement
 - Caractères spéciaux du Shell
 - Entrées/Sorties et redirection

Entrée/Sortie standard d'un processus

Un processus (programme en execution) dispose de trois fichiers :

- Entrée standard pour recevoir les arguments, parametres, données
- Sortie standard : pour afficher ses resultats (`printf` écrit dans ce fichier),
- Sortie des erreurs (`fprintf(stderr, ...)` écrit dans ce fichier)

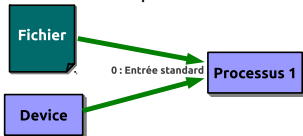
Ces fichiers sont représentés dans le système par des numéros : leurs descripteurs de fichiers



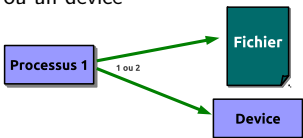
Redirection d'entrée/Sortie

La redirection est l'opération qui consiste à placer l'entrée ou la sortie standard vers un fichier autre que celui par défaut

- Redirection d'entrée : écrire le contenu d'un fichier ou device dans l'entrée standard d'un processus



- Redirection de sortie : rediriger la sortie standard d'un processus vers un fichier ou un device



Redirection d'entrée/Sortie

Exemples :

- redirection d'entrée <

```
#> cat < file.txt
```

- redirection de sortie >(vider le fichier de sortie),» (ajouter à la fin)

```
#> ls -l > filelog
#> cat file1 file2 > file3
#> ps aux >> filelog
```

- redirection de sortie des erreurs

```
#> ls -l 2> errors
#> ps aux 2>> errors
```

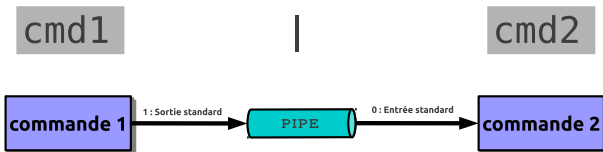


Les tubes (pipes) de commandes : |

le pipe (caractère barre verticale |) permet d'injecter la sortie d'une commande sur l'entrée standard d'une autre commande.

Forme générale :

```
expression_1 | expression_2 | ... expression_n
```



Exemples

```
> ls | grep 'mp3'
> ls | grep 'mp3' | sort -r
```



Bibliographie



Carla Schroder

Linux Cookbook : Practical Advice for Linux System Administrators

O'Reilly, November 2004



Richard Blum,

Linux Command Line and Shell Scripting Bible,

Wiley Publishing, Inc. 2008



Michael Palmer

Guide to UNIX Using Linux, FOURTH EDITION

Course Technology, Cengage Learning, 2008



Credits

Cette présentation a été réalisée avec des outils libres :

la classe BEAMER de $\text{\LaTeX}$



Libreoffice Draw

